

คู่มือการใช้งาน Fable2Opus

สอน Claude Opus 4.8 ให้ทำงานด้วย "กระบวนการ" ระดับ Fable 5 — ติดตั้งครั้งเดียว ใช้ได้ทุกโปรเจกต์

เวอร์ชัน 1.3 · กรกฎาคม 2026 ติดตั้งด้วยคำสั่งเดียว ไม่ต้องมีพื้นฐานลึก

1 โปรเจกต์นี้คืออะไร

Fable2Opus (F2O) คือชุด "ถ่ายทอดกระบวนการทำงาน" — เกิดจากการวิเคราะห์ว่า Claude Fable 5 (รุ่นท็อป) ทำงานต่างจาก Claude Opus 4.8 ตรงไหน แล้วกลั่นความต่างส่วนที่เป็น**วินัยและขั้นตอน** (ไม่ใช่ความฉลาดดิบ) ออกมาเป็นกติกา+คู่มือที่ Opus 4.8 ทำตามได้จริง ผลคือได้ Opus ที่ทำงานรอบคอบระดับใกล้เคียงรุ่นท็อป ในราคาถูกกว่าครึ่งหนึ่งและ limit การใช้งานที่เยอะกว่า

ทำอะไรได้บ้าง

- ✓ **งานคิดทุกชนิด** — แก่บ้าง สร้าง feature, refactor, migration แบบมีหลักฐานทุกขั้น
- ✓ **ล่าบักยาก** — บักเป็นๆ หายๆ, race condition ด้วยวิธีเชิงสถิติ ไม่ใช่เดา
- ✓ **Code review** — หาให้ครบก่อนกรอง ไม่ปล่อยบักหลุด เพราะถูกสั่งให้ "เอาแต่เรื่องใหญ่"
- ✓ **งานยาวข้ามวัน/ข้ามsession** — จดโน้ตกันงานหล่นเมื่อ context เต็ม
- ✓ **งานเอกสาร** — รายงาน spreadsheet slide ที่ตัวเลขตรวจทานแล้วทุกตัว
- ✓ **อ่านค่าจากภาพ/สไลด์** — ตาราง ตัวเลข แบบทวนสอบไม่เผลอจุดที่มองไม่ชัด
- ✓ **งานใหญ่แบบขนาน** — แดกงานให้ sub-agent คลายตัวช่วยแล้วตรวจรวม
- ✓ **กันหลอกว่าเสร็จ** — hook บล็อกคำว่า "done" ที่ไม่มีหลักฐานจริง

2 ใช้ F2O กับไม่ใช่ ต่างกันอย่างไร

สถานการณ์	Opus 4.8 เพียงๆ (พฤติกรรมตั้งต้นที่ Anthropic บันทึกรไว้)	Opus 4.8 + F2O
บักเป็นๆ หายๆ	✗ เทสต์ผ่าน 1 รอบก็ประกาศ "fixed"	✓ บังคับรันซ้ำ 10–30 รอบ รายงานตัวเลขก่อน/หลังแก้
คำว่า "เสร็จแล้ว"	✗ ต้องเชื่อคำพูด ไม่รู้ตรวจจริงไหม	✓ ทุก claim ชี้หลักฐานได้ · แยก VERIFIED / NOT VERIFIED ชัด · hook บล็อกถ้าโกหก
คำถามระหว่างงาน	✗ ถามเรื่องเล็กน้อย จบเทิร์นด้วย "เอาไหมครับ?"	✓ เรื่องย้อนกลับได้ตัดสินใจเอง+จดไว้ให้ veto ที่หลัง — งานจบในเทิร์นเดียวบ่อยขึ้น
ข้อเท็จจริง เวอร์ชัน/API/ราคา	✗ ตอบจากความจำ (อาจล้าสมัย)	✓ ค้นเว็บ/อ่านไฟล์จริงก่อนพูดเสมอ

ขอบเขตการแก้ไข	❌ ซ่อม refactor แลเม diff บวม	✅ diff ตรงใจทย์ ปัญหาข้างเคียงถูก "จดยางาน" แทนแก้มั่ว
Review ที่สั่งว่า "เอาเฉพาะเรื่องใหญ่"	❌ ทำตามตรงตัวจนตกหล่นบัก (recall ร่วง — Anthropic วดเอง)	✅ หาให้หมดก่อน แล้วค่อยกรองตอนรายงาน
งานหลาย requirement	❌ ข้อท้ายๆ ซ่อมหล่นหาย	✅ checklist คุมทุกข้อ ปิดงานเทียบหลักฐานรายข้อ (n/n)
งานยาว / ทำต่อวันหลัง	❌ สัมบริบท ทำซ้ำหรือหล่นหลัง context เต็ม	✅ โน้ต WORKING_NOTES.md + พรีฟิบบริบทจากหลักฐานจริง

เพื่อความตรงไปตรงมา: คอลัมน์ซ้ายคือพฤติกรรมตั้งต้นของ Opus 4.8 ตามเอกสาร migration ของ Anthropic เอง ส่วนขวาคือโลกที่ F2O บังคับ · หลักฐานเชิงตัวเลขที่มีแล้ว: instruction แบบเดียวกันนี้ลดการถามพรีาเพรื่อ ~12 จุด (Anthropic วด) และการทดสอบ A/B ของโปรเจกต์พบการรายงานปัญหาข้างเคียงเพิ่มจาก 1/5 → 5/5 เมื่อโหลด skill · สิ่งที่ F2O ให้ไม่ได้: ความฉลาดดิบ — ช่องว่างด้าน insight ยังเป็นของรุ่นท็อป

3 ส่วนประกอบ 3 ชั้น

ชั้น	คืออะไร	จำเป็นไหม
Layer 0	กฎแกน 2 ไฟล์ (<code>CLAUDE.md</code> + <code>constants.md</code>) — ห้ามบอกว่า "เสร็จ" โดยไม่มีหลักฐาน · ไม่ถามเรื่องเล็กน้อย · จดโน้ตไฟล์เดียว	จำเป็น
Layer 1	Skills 13 ตัว — คู่มือเชิงลึกที่ Claude โหลดเองอัตโนมัติ ตามสถานการณ์ (แก้บัก → ตัว debug ทำงาน, จะสรุปงาน → ตัวรายงานทำงาน)	จำเป็น
Layer 2	Hook ตรวจจับ — สคริปต์ที่ บล็อก ข้อความ "เสร็จแล้ว" ที่ไม่มีหลักฐานจริง ก่อนถึงมือคุณ	ทางเลือก (แนะนำ)

สิ่งที่เปลี่ยน

- ✅ ค้น/อ่านโค้ดก่อนแก้ ไม่เดาจากความจำ
- ✅ รับเคสดีซ้ำก่อนพูดว่า "fixed" (บักแบบเป็นๆ หายๆ รับซ้ำ 10–20 รอบ)
- ✅ ทำงานจนจบเทิร์น ไม่ถาม "เอาไหมครับ?" กลางทาง
- ✅ สรุปงานแบบ VERIFIED / NOT VERIFIED / ASSUMED

สิ่งที่ไม่เปลี่ยน

ความฉลาดดิบของโมเดล — ชุดนี้ถ่ายทอด**วินัยการทำงาน** ไม่ใช่สมองเสริม งานที่ต้องใช้ insight ลึกยังเป็นขีดจำกัดของตัวโมเดลเช่นเดิม

4 สิ่งที่ต้องมีก่อน

- ✅ **Claude Code** (แอป desktop หรือ CLI) ที่ล็อกอินใช้งานได้แล้ว
- ✅ **โมเดล Opus 4.8** — ใน session พิมพ์ `/model` แล้วเลือก Opus (ชุดนี้ออกแบบเพื่อ Opus 4.8 แต่ใช้กับโมเดลอื่นก็ไม่พัง)
- ✅ **Python 3** — เฉพาะถ้าติดตั้ง Layer 2 (macOS มีอยู่แล้ว — เช็ค: `python3 --version`)

แหล่งข้อมูล: GitHub github.com/Work-in-Problem/F20 (open source, MIT) · เว็บไซต์ f2o.workinproblem.com — ติดตั้งด้วยคำสั่งเดียวได้ 2 ช่องทาง (ดูหัวข้อ 5) หรือ clone มาติดตั้งมือ

5 ติดตั้ง

ทาง A · แนะนำ Claude Code plugin — ครอบคลุมชั้น: skills + กฎเกณฑ์อัตโนมัติ + hook

```
claude plugin marketplace add Work-in-Problem/F20
claude plugin install f2o@f2o
```

หรือพิมพ์ `/plugin` ในแอปแล้วเลือก marketplace [Work-in-Problem/F20](https://github.com/Work-in-Problem/F20) — เปิดใช้แล้วกฎ Layer 0 ถูกจัดเข้าทุก session อัตโนมัติ · ควบคุม: `/f2o:off` (soft-off — สกิลยังใช้ได้) · `/f2o:on` · `/f2o:status` · shorthand: `/f2o:vbc` ฯลฯ 13 ตัว

ทาง B `npx skills` — ได้เฉพาะ skills 13 ตัว

```
npx skills add Work-in-Problem/F20 --all -y -g
```

ช่องทางนี้ไม่พกกฎเกณฑ์ (Layer 0) กับ hook — copy เองตามทาง C ชั้น 2-3

ทาง C — ติดตั้งมือจาก clone (~2 นาที)

เปิดแอป **Terminal** แล้ว copy คำสั่งไปวางทีละกล่อง

ขั้น 1 ติดตั้ง Skills 13 ตัว แบบ global — ทำครั้งเดียว ใช้ได้ทุกโปรเจกต์

```
git clone https://github.com/Work-in-Problem/F20.git && SRC="$PWD/F20"

mkdir -p ~/.claude/skills
cp -R "$SRC/skills/" ~/.claude/skills/

# ตรวจสอบเข้าครบ — ต้องเห็น 13 โฟลเดอร์
ls ~/.claude/skills
```

ขั้น 2 ติดตั้ง Layer 0 ลง "โปรเจกต์ที่จะใช้งาน" — ทำซ้ำกับทุกโปรเจกต์ที่ต้องการ

```
cd /path/to/your-project          # ← แก้เป็นโปรเจกต์จริงของคุณ

cp "$SRC/core/constants.md" ./constants.md

# ถ้าโปรเจกต์ยังไม่มีไฟล์ CLAUDE.md:
cp "$SRC/core/CLAUDE-core.md" ./CLAUDE.md

# ถ้ามี CLAUDE.md อยู่แล้ว ให้ต่อท้ายแทน:
cat "$SRC/core/CLAUDE-core.md" >> ./CLAUDE.md
```

ขั้น 3 · ทางเลือก ติดตั้ง Hook กันหลอกว่าเสร็จ (Layer 2)

```
mkdir -p .claude/hooks
cp "$SRC/hooks/claim_gate.py" .claude/hooks/
cat > .claude/settings.json <<'JSON'
{"hooks":{"Stop":[{"hooks":[{"type":"command","command":"python3",
  "args":["${CLAUDE_PROJECT_DIR}/.claude/hooks/claim_gate.py"],"timeout":30}]}}}
JSON
```

ระวัง: ถ้าโปรเจกต์มี `.claude/settings.json` อยู่แล้ว **อย่า copy** กับ — เปิดไฟล์แล้วเพิ่มเฉพาะก้อน `"hooks"` จากไฟล์ต้นทางเข้าไปแทน

6 เรียกใช้ยังไง

คำตอบสั้น: ไม่ต้องเรียก — มันทำงานเอง

- `cd` เข้าโปรเจกต์ที่ติดตั้งแล้ว → เปิด `claude` → พิมพ์ `/model` เลือก **Opus 4.8**
- สั่งงานตามปกติ — Claude โหลด skill ที่ตรงสถานการณ์เองอัตโนมัติ

คุณสั่งว่า...	Skill ที่ทำงานเอง
"แก้บั๊ก login แล้วทดสอบให้ผ่าน"	<code>root-causing-bugs</code> + <code>verifying-before-claiming</code>
"รีวิว PR นี้น้อย"	<code>reviewing-code</code>
"ทำรายงานยอดขายจากไฟล์ csv นี"	<code>producing-deliverables</code>

อยากเรียกเองเจาะจงก็ได้: พิมพ์ `/` แล้วเลือกชื่อ skill หรือบอกตรงๆ ว่า *"ใช้ skill `scoping-code-changes` ด้วย"*

เคล็ดลับสำคัญกับ Opus 4.8: ให้ **โจทย์ครบตั้งแต่ข้อความแรก** — Opus ทำงานยาวได้ดีที่สุดเมื่อได้ spec เต็มในเทิร์นเดียว อย่าป้อนทีละนิด แล้วปล่อยให้ทำงานจบ ไม่ต้องจ้ำจี้ระหว่างทาง

7 รู้ว่าติดตั้งสำเร็จจากอะไร

เปิด session ใหม่แล้วลองสั่ง: *"แก้ typo ในไฟล์ X แล้วบอกด้วยว่า verify ยังไง"* — สัญญาณที่ควรเห็น:

- ✓ อ่านไฟล์ก่อนแก้ และ**ไม่ทำพิธีใหญ่โต**กับงานจ๊ว (กตึกา proportionality ทำงาน)
- ✓ สรุปปิดงานขึ้นต้นด้วยผลลัพธ์ 1 บรรทัด + หมวด VERIFIED พร้อมคำสั่งที่รันจริง
- ✓ ไม่จบเทิร์นด้วย "Want me to also...?" สำหรับสิ่งที่ควรทำต่อเอง
- ✓ เช็กแบบบกล็อก (ทาง B/C): `ls ~/.claude/skills | wc -l` ต้องได้ **13** · ทาง A (plugin): `claude plugin list` ต้องเห็น `f2o` enabled

8 ปัญหาพบบ่อย

อาการ	ทางแก้
-------	--------

Skill ไม่โหลด / พิมพ์ / แล้วไม่เห็น	ปิด-เปิด session ใหม่ (skills โหลดตอนเริ่ม session) · ตรวจสอบ <code>ls ~/.claude/skills</code>
Hook บล็อกทั้งที่ไม่ควรบล็อก	ปิดชั่วคราว: <code>export FABLE20PUS_HOOKS_OFF=1</code> แล้วเปิด session ใหม่
ทำพิธีเยอะเกินไปกับงานเล็ก	บอกตรงๆ ได้เลย ("งานนี้เล็ก ข้ามพิธี") — ทุกกฎมีช่องยกเว้นพร้อมเหตุผล 1 บรรทัดอยู่แล้ว
ใช้ token มากขึ้น	ปกติ — งาน verify หนักใช้ ~1.3–2 เท่า แลกกับความน่าเชื่อถือ งานเล็กแทบไม่ต่าง
เลขในกฎขัดกันเอง	<code>constants.md</code> ชนะเสมอ (กติกา Layer 0 ข้อ 4)

9 อัปเดต / ก่อนการติดตั้ง

ต้องการ	วิธีทำ
อัปเดต	แก้ที่ไฟล์ดอร์ต้นทาง แล้วรับคำสั่ง copy ขึ้น 1–3 ชั้น (ทับของเก่า)
ถอน skills	ลบไฟล์ดอร์ที่ไม่ต้องการออกจาก <code>~/.claude/skills/</code> (เลือกลบรายตัวได้)
ถอน Layer 0	ลบ <code>constants.md</code> และส่วน Fable2Opus ใน <code>CLAUDE.md</code> ของโปรเจกต์
ถอน hook	ลบก่อน <code>hooks</code> ใน <code>.claude/settings.json</code> + ลบไฟล์ดอร์ <code>.claude/hooks/</code>

10 Skills ทั้ง 13 ตัว — ใครทำอะไร โหลดเมื่อไหร่

Skill	ทำอะไร	โหลดเองเมื่อ
verifying-before-claiming	พิสูจน์ก่อนพูดว่าเสร็จ: ทำบักให้เกิดขึ้นก่อนแก้ · รับ flow จริง · อ่าน output ไม่ใช่แค่ exit code	ทำ feature / แก้บัก / จะพูดว่า done
finishing-the-turn	ไม่ถามเรื่องเล็ก · ทำงานจนจบเทิร์น · ห้ามจบด้วยคำสัญญา "เดี๋ยวจะ..."	ก่อนถามคำถาม / จบเทิร์น / เจอ error
search-first-context	ค้น/อ่านก่อนพูดหรือแก้ — เวอร์ชัน, API, ราคา ต้องเช็คก่อน ไม่เอาจากความจำ	เริ่มงานใน codebase / จะพูดข้อเท็จจริง
planning-to-done	แตกโจทย์เป็น checklist ครบทุกข้อก่อนลงมือ · ปิดงานด้วยหลักฐานรายชื่อ	งานหลายส่วน / feature / migration
scoping-code-changes	diff เล็กตรงโจทย์ · ไม่ refactor แรก · เจอปัญหาข้างเคียง = จดรายงาน ไม่แก้เอง	ก่อนแก้โค้ดใดๆ
root-causing-bugs	debug แบบตั้งสมมติฐาน-ทดลอง หา root cause จริง ไม่ปะ band-aid · จัดการ flaky test แบบสถิติ	มีบัก / test พัง / อาการ "เป็นบางครั้ง"
reviewing-code	รีวิวแบบ "หาให้หมดก่อน ค่อยกรอง" · ตรวจสอบทุก finding กับ โค้ดจริงก่อนรายงาน	รีวิว PR / diff / หา bug ในโค้ด
delegating-parallel-work	แตกงานใหญ่ให้ sub-agent ทำขนานกัน แล้วตรวจสอบผลรวมเองก่อนเชื่อ	งาน 3+ ส่วนอิสระ / หลาย package
managing-working-memory	จดโน้ต WORKING_NOTES.md ให้งานยาวรอดข้าม session และ context compaction	งานยาว / ทำต่อวันหลัง / หลัง compact
outcome-first-reporting	สรุปแบบผลลัพธ์ขึ้นก่อน · มีหมวด "ยังไม่เสร็จ" ชัดเจน · ไม่เล่านิทานยืดยาว	ตอนเขียนสรุป / ตอบสถานะ
producing-deliverables	ทำรายงาน/spreadsheet/slide แบบตรวจข้อมูลก่อนคำนวณ + เปิดไฟล์ทวนก่อนส่ง	งานที่ผลลัพธ์เป็นเอกสาร ไม่ใช่โค้ด
extracting-from-images	อ่านค่าจากภาพ/สแกนแบบ crop-ซูม-ทวนสอบ ไม่เดาตัวเลขที่มองไม่ชัด	ต้องดึงตัวเลข/ตารางจากรูปภาพ
inventorying-capabilities	สำรวจเครื่องมือ (MCP/skills) ที่มีจริงก่อนบอกว่า "ทำไม่ได้" หรือลงมือแบบอ้อมโลก	session ที่เชื่อมต่อเครื่องมือพิเศษ